Structs and functions practice

Submit Assignment

Due No Due Date **Points** 4 **Submitting** a file upload

1. Overview

In this assignment, you will demonstrate your understanding of **structures**, **pointers**, and **dynamic memory allocation** (heap memory).

Your task is to implement a library that handles integer matrices of arbitrary dimensions. Because the size of these matrices is not known at compile-time, you must manage memory manually using `malloc` and `free`.

2. The Matrix Structure

You must define a struct to represent the matrix. This struct should not store the integers directly, but rather store the dimensions and a pointer to the memory area where the integers are kept.

Define your struct as follows:

```
typedef struct {
    int rows;
    int cols;
    int *data;
} Matrix;
```

> **Note on implementation:** You should store the matrix elements in a **1D array** (flattened). You will need to use pointer arithmetic to map a 2D coordinate (row, col) to a 1D index.

3. Required Functions

You are required to implement the following functions. Do not modify the function signatures.

3.1 Allocation and Deallocation

`Matrix* init_matrix(int n, int m)`

- **Purpose:** Allocates memory for a new matrix of size $n \times m$.
- **Parameters:**
  - `n`: The number of rows.
  - `m`: The number of columns.
- **Behavior:**
  - Allocates the `Matrix` structure.
  - Allocates the `data` array of size n * m.

- Initializes all data elements to `0`.
- Returns `NULL` if allocation fails or if dimensions are invalid (<1).

`void free_matrix(Matrix* mat)`

- **Purpose:** Frees all memory associated with the matrix.
- **Behavior:**
  - Frees the internal `data` array.
  - Frees the `Matrix` structure itself.
  - Should safely handle `NULL` input.

3.2 Data Access

`int* get_element(Matrix* mat, int row, int col)`

- **Purpose:** Returns a pointer to the element at the specified index.
- **Behavior:**
  - **Bounds Checking:** You must check if `row` and `col` are within valid ranges (0 <= row < n and 0 <= col < m).
  - If the index is invalid, print an error message to `stderr` and return `NULL`.
  - If valid, return a pointer to the specific integer in the data array.
- **Why a pointer?** Returning `int*` allows the user to read *and* write the value.
  - *Usage:* `*get_element(mat, 0, 0) = 10;`

3.3 Operations

`Matrix* multiply_matrices(Matrix* a, Matrix* b)`

- **Purpose:** Computes the matrix product of A and B.
- **Behavior:**
  - Checks if the dimensions match for multiplication (Columns of A must equal Rows of B).
  - Allocates a **new** matrix for the result.
  - Computes the values using the standard matrix multiplication formula as learned in Basic Mathematics.
  - Returns the new result matrix (or `NULL` if dimensions are incompatible).

4. Usage Example

Your code should work with a `main` function similar to the one below. You should create your own `main.c` to test your library thoroughly.

C

```c
#include <stdio.h>
#include "matrix.h"

int main() {
    // 1. Create a 2x3 matrix
    Matrix* m1 = init_matrix(2, 3);

    // 2. Set values using the indexer
    if (m1 != NULL) {
        *get_element(m1, 0, 0) = 1;
        *get_element(m1, 0, 1) = 2;
        *get_element(m1, 0, 2) = 3;
        *get_element(m1, 1, 0) = 4;
        *get_element(m1, 1, 1) = 5;
        *get_element(m1, 1, 2) = 6;
    }

    // 3. Create a 3x2 matrix
    Matrix* m2 = init_matrix(3, 2);
    // ... initialize m2 values ...

    // 4. Multiply
    Matrix* result = multiply_matrices(m1, m2);

    // 5. Clean up
    free_matrix(m1);
    free_matrix(m2);
    free_matrix(result);

    return 0;
}
```

5. Submission Requirements

- **Files:** Submit `matrix.c and` `matrix.h`.
- **Memory Leaks:** Your code must run without memory leaks.
- Compilation: Your code must compile with no warnings using:

  gcc -Wall -Wextra matrix.c main.c -o matrix_program